

1. Customer Activity Report

Management wants to see **all transactions along with the customer's name and city** to understand where activity is happening

```
SELECT t.transaction_id,t.amount, t.transaction_date,t.transaction_type, c.customer_name, c.city
FROM transactions t
JOIN customers c
ON t.customer_id=c.customer_id
```

```
42 -- Q1.Customer Activity Report
43 -- Management wants to see all transactions along with the customer's name and city to understand where activity is happening
44
45 • SELECT t.transaction_id, c.customer_name, c.city,t.amount, t.transaction_date,t.transaction_type
46 FROM transactions t
47 JOIN customers c
48 ON t.customer_id=c.customer_id
49
```

10%	88:51					
Result Grid						
Filter Rows: Search Export:						
transaction_id	customer_name	city	amount	transaction_date	transaction_type	
1	Alice	Toronto	500.00	2025-01-01	deposit	
2	Alice	Toronto	-200.00	2025-01-02	withdrawal	
3	Bob	Vancouver	1000.00	2025-01-01	deposit	
4	Charlie	Toronto	-300.00	2025-01-03	withdrawal	
5	Alice	Toronto	-100.00	2025-01-04	withdrawal	

2. Customer Value Analysis

The finance team wants to know **how much money each customer has transacted in total**

```
SELECT c.customer_name, SUM(t.amount) AS total_transacted_money
FROM customers c
LEFT JOIN transactions t
ON c.customer_id=t.customer_id
GROUP BY c.customer_id, c.customer_name
```

```

51  -- Q2.Customer Value Analysis
52  -- The finance team wants to know how much money each customer has transacted in total
53
54  SELECT c.customer_name, SUM(t.amount) AS total_transacted_money
55  FROM customers c
56  LEFT JOIN transactions t
57  ON c.customer_id=t.customer_id
58  GROUP BY c.customer_id, c.customer_name
59

```

00% 1:49 1 error found

Result Grid Filter Rows: Search Export:

customer_name	total_transacted_money
Alice	200.00
Bob	1000.00
Charlie	-300.00
David	NULL

3. Customer Engagement

The marketing team wants to know **how many transactions each customer has made** so they can identify highly engaged users.

```

SELECT c.customer_id, c.customer_name, count(t.transaction_id) AS total_transactions
FROM customers c
LEFT JOIN transactions t
ON c.customer_id=t.customer_id
GROUP BY c.customer_id, c.customer_name

```

```

61  -- Q3.Customer Engagement
62  -- The marketing team wants to know how many transactions each customer has made so they can identify highly engaged users.
63
64  SELECT c.customer_id, c.customer_name, count(t.transaction_id) AS total_transactions
65  FROM customers c
66  LEFT JOIN transactions t
67  ON c.customer_id=t.customer_id
68  GROUP BY c.customer_id, c.customer_name
69
70

```

100% 40:68 1 error found

Result Grid Filter Rows: Search Export:

customer_id	customer_name	total_transactions
1	Alice	3
2	Bob	1
3	Charlie	1
4	David	0

4. Dormant Customers

The retention team wants to identify **customers who signed up but have never made a transaction.**

```
SELECT c.customer_name, c.customer_id, c.city, c.signup_date
FROM customers c
LEFT JOIN transactions t
ON c.customer_id=t.customer_id
WHERE t.transaction_id IS NULL
```

```
70  -- Dormant Customers
71  -- The retention team wants to identify customers who signed up but have never made a transaction.
72
73  SELECT c.customer_name, c.customer_id, c.city, c.signup_date
74  FROM customers c
75  LEFT JOIN transactions t
76  ON c.customer_id=t.customer_id
77  WHERE t.transaction_id IS NULL
78
```

100%

31:76

1 error found

Result Grid

Filter Rows:

Q

Search

Export:

customer_name	customer_id	city	signup_date
David	4	Montreal	2024-04-20

5. Withdrawal Behavior

Risk management wants to know **which customers have made withdrawals**, since withdrawals may indicate liquidity needs or account usage

```
SELECT c.customer_id, c.customer_name, c.city, t.transaction_type
FROM customers c
JOIN transactions t
ON c.customer_id=t.customer_id
WHERE t.transaction_type = 'withdrawal'
```

```

80  -- Q5.Withdrawal Behavior
81  -- Risk management wants to know which customers have made withdrawals, since withdrawals may indicate liquidity needs or account usage
82
83  SELECT c.customer_id, c.customer_name, c.city, t.transaction_type
84  FROM customers c
85  JOIN transactions t
86  ON c.customer_id=t.customer_id
87  WHERE t.transaction_type = 'withdrawal'
88
89

```

100% 8:80 1 error found

Result Grid Filter Rows: Search Export:

customer_id	customer_name	city	transaction_type
1	Alice	Toronto	withdrawal
3	Charlie	Toronto	withdrawal
1	Alice	Toronto	withdrawal

6. Deposit Contribution

The treasury team wants to understand **how much money each customer has deposited in total**.

```

SELECT c.customer_id, c.customer_name, SUM (t.amount) AS total_deposited_amount
FROM customers c
JOIN transactions t
ON c.customer_id=t.customer_id
WHERE t.transaction_type = 'deposit'
GROUP BY c.customer_id, c.customer_name

```

```

89  -- Q6.Deposit Contribution
90  -- The treasury team wants to understand how much money each customer has deposited in total
91
92  SELECT c.customer_id, c.customer_name,SUM(t.amount) AS total_deposited_amount
93  FROM customers c
94  JOIN transactions t
95  ON c.customer_id=t.customer_id
96  WHERE t.transaction_type = 'deposit'
97  GROUP BY c.customer_id, c.customer_name
98
99

```

100% 40:97 1 error found

Result Grid Filter Rows: Search Export:

customer_id	customer_name	total_deposited_amo...
1	Alice	500.00
2	Bob	1000.00

7. High-Risk Spending Customers

Fraud monitoring wants to identify **customers who have withdrawn the largest total amount of money**.

```
SELECT c.customer_id, c.customer_name, ABS(SUM(t.amount))AS total_amount_withdrawn
FROM customers c
JOIN transactions t
ON c.customer_id=t.customer_id
WHERE t.transaction_type = 'withdrawal'
GROUP BY c.customer_id, c.customer_name
ORDER BY total_amount_withdrawn DESC
```

```
99  -- Q7. High-Risk Spending Customers
100 -- Fraud monitoring wants to identify customers who have withdrawn the largest total amount of money.
101
102 SELECT c.customer_id, c.customer_name, ABS(SUM(t.amount))AS total_amount_withdrawn
103 FROM customers c
104 JOIN transactions t
105 ON c.customer_id=t.customer_id
106 WHERE t.transaction_type = 'withdrawal'
107 GROUP BY c.customer_id, c.customer_name
108 ORDER BY total_amount_withdrawn DESC
109
```

100%	40:106	1 error found
Result Grid	Filter Rows: Search	Export:
customer_id	customer_name	total_amount_withdra...
1	Alice	300.00
3	Charlie	300.00

8. Regional Revenue Analysis

Leadership wants to know **how much total transaction value is generated from each city**

```
SELECT c.city, SUM(t.amount) AS total_transaction_value
FROM customers c
LEFT JOIN transactions t
ON c.customer_id=t.customer_id
GROUP BY c.city
```

```

110 -- Q8. Regional Revenue Analysis
111 -- Leadership wants to know how much total transaction value is generated from each city
112
113 SELECT c.city, SUM(t.amount) AS total_transaction_value
114 FROM customers c
115 LEFT JOIN transactions t
116 ON c.customer_id=t.customer_id
117 GROUP BY c.city

```

100% 17:114 1 error found

Result Grid Filter Rows: Search Export:

city	total_transaction_value
Toronto	-100.00
Vancouver	1000.00
Montreal	NULL

9. Customer Transaction Patterns

The analytics team wants to calculate the **average transaction amount for each customer**.

```

SELECT c.customer_id, c.customer_name, AVG( t.amount) AS average_amount
FROM customers c
JOIN transactions t
ON c.customer_id=t.customer_id
GROUP BY c.customer_id, c.customer_name

```

```

119 -- Q9. Customer Transaction Patterns
120 -- The analytics team wants to calculate the average transaction amount for each customer.
121
122 SELECT c.customer_id, c.customer_name, AVG( t.amount) AS average_amount
123 FROM customers c
124 JOIN transactions t
125 ON c.customer_id=t.customer_id
126 GROUP BY c.customer_id, c.customer_name
127
128

```

100% 41:126 1 error found

Result Grid Filter Rows: Search Export:

customer_id	customer_name	average_amount
1	Alice	66.666667
2	Bob	1000.000000
3	Charlie	-300.000000

10. Customer Onboarding Effectiveness

Product managers want to know **when each customer made their first transaction after signing up**

```
SELECT c.customer_id, c.customer_name, c.signup_date, MIN(t.transaction_date) AS
first_transaction_date
FROM customers c
JOIN transactions t
ON c.customer_id=t.customer_id
GROUP BY c.customer_id, c.customer_name
```

```
128  -- Q10. Customer Onboarding Effectiveness
129  -- Product managers want to know when each customer made their first transaction after signing up
130
131  SELECT c.customer_id, c.customer_name, c.signup_date, MIN(t.transaction_date) AS first_transaction_date
132  FROM customers c
133  JOIN transactions t
134  ON c.customer_id=t.customer_id
135  GROUP BY c.customer_id, c.customer_name
136
```

100% 41:135 1 error found

Result Grid Filter Rows: Search Export:

	customer_id	customer_name	signup_date	first_transaction_d...
1		Alice	2024-01-10	2025-01-01
2		Bob	2024-02-15	2025-01-01
3		Charlie	2024-03-05	2025-01-03

11. Large Transaction Monitoring

Compliance wants a report showing **customers who have made transactions larger than \$500**

```
SELECT c.customer_id, c.customer_name, t.amount, t.transaction_type, t.transaction_date
FROM customers c
JOIN transactions t
ON c.customer_id=t.customer_id
WHERE ABS(t.amount) > 500
```



```

137 -- Q11. Large Transaction Monitoring
138 -- Compliance wants a report showing customers who have made transactions larger than $500
139
140 SELECT c.customer_id, c.customer_name, t.amount, t.transaction_type, t.transaction_date
141 FROM customers c
142 JOIN transactions t
143 ON c.customer_id=t.customer_id
144 WHERE ABS(t.amount) > 500
145

```

100% 26:144 1 error found

Result Grid					
Filter Rows: Search					
Export:					
customer_id	customer_name	amount	transaction_type	transaction_date	
2	Bob	1000.00	deposit	2025-01-01	

12. Market Penetration by City

The growth team wants to know **how many customers in each city are actively transacting.**

```

SELECT c.city, COUNT(distinct c.customer_id) AS active_customers
FROM customers c
JOIN transactions t
ON c.customer_id=t.customer_id
GROUP BY c.city

```

```

146 -- Q12. Market Penetration by City
147 -- The growth team wants to know how many customers in each city are actively transacting.
148
149 SELECT c.city, COUNT(distinct c.customer_id) AS active_customers
150 FROM customers c
151 JOIN transactions t
152 ON c.customer_id=t.customer_id
153 GROUP BY c.city
154
155

```

100% 16:153 1 error found

Result Grid		
Filter Rows: Search		
Export:		
city	active_custom...	
Toronto	2	
Vancouver	1	

13. Recent Customer Activity

Customer success wants to see **the most recent transaction for each customer** to understand who is still active.

```
SELECT c.customer_id, c.customer_name, MAX(t.transaction_date) AS recent_transaction_date
FROM customers c
JOIN transactions t
ON c.customer_id=t.customer_id
GROUP BY c.customer_id, c.customer_name
```

```
-- .
155 -- Q13. Recent Customer Activity
156 -- Customer success wants to see the most recent transaction for each customer to understand who is still active.
157
158 SELECT c.customer_id, c.customer_name, MAX(t.transaction_date) AS recent_transaction_date
159 FROM customers c
160 JOIN transactions t
161 ON c.customer_id=t.customer_id
162 GROUP BY c.customer_id, c.customer_name
163
164
```

100% 20:160 1 error found

Result Grid Filter Rows: Search Export:

	customer_id	customer_name	recent_transaction_d...
1	Alice	2025-01-04	
2	Bob	2025-01-01	
3	Charlie	2025-01-03	

14. High Deposit Customers

Finance wants to identify **customers whose total deposits exceed \$1000**, as these customers may qualify for premium services.

```
SELECT c.customer_id, c.customer_name, SUM(t.amount) AS total_deposits
FROM customers c
JOIN transactions t
ON c.customer_id=t.customer_id
WHERE t.transaction_type = 'deposit'
GROUP BY c.customer_id, c.customer_name
HAVING SUM(t.amount) > 1000
```

[illegible]

100% 36:183 1 error found

Result Grid Filter Rows: Search Export:

	transaction_date	city	daily_transactions_total
	2025-01-01	Toronto	500.00
	2025-01-02	Toronto	-200.00
	2025-01-01	Vancouver	1000.00
	2025-01-03	Toronto	-300.00
	2025-01-04	Toronto	-100.00